# ARGUS

## From Visual Streams to Autonomous Data Products

Technical whitepaper | Version 0.1 | 26 October 2026

**Abstract.** ARGUS is a proposed visual intelligence platform that ingests video, images and contextual data; converts visual signals into structured observations; and composes reusable, continuously updated data products with AI-directed workflows. Rather than treating each camera as a separate application, ARGUS treats observations as shared infrastructure from which mapping, analytics, event monitoring and decision support can be derived.

**Thesis.** AI agents should discover requirements, choose suitable models and transformations, and explain evidence. Deterministic workers should execute arithmetic, filtering, time windows, spatial operations and publication. Agent outputs must remain testable, attributable and reviewable.

**Status.** Proposed architecture, not an experimentally validated system. Numerical examples are illustrative.

## 1. What qualifies as a data product?

A data product is a reusable, documented and accessible data capability that serves a consumer need. It is not merely a chart, CSV file or model inference. ARGUS products have a stable identity, output contract, freshness objective, lineage, quality checks, security policy and delivery interface. [1][2]

| Attribute | ARGUS requirement |
|---|---|
| Identity | product_id, version, owner, domain and description |
| Contract | input/output schemas, units, coordinate reference, time semantics |
| Evidence | source IDs, media references, model and transformation versions |
| Quality | confidence calibration, coverage, known limitations, validation status |
| Operations | update frequency, latency/freshness objective, failure policy |
| Access | project authorization, privacy rules, retention and export permissions |

## 2. The transformation ladder

| Level | Example | Meaning |
|---|---|---|
| L0 - Media | Camera footage | Unstructured source, retained under policy |
| L1 - Detection | Car bounding box | Localized and classified observation |
| L2 - Tracking | Vehicle track | Time-linked observations within a camera |
| L3 - Event | Vehicle crossing line | Domain event with evidence and timestamp |
| L4 - Measurement | 97 cars / 15 minutes | Validated count with window and coverage |
| L5 - Product | Traffic flow dataset + API | Versioned, reusable, continually maintained output |
| L6 - Decision | Investigate queue increase | Interpretation with assumptions and human review |

Avoid promoting an inference into a ground truth fact. Detection confidence, data coverage and domain interpretation are different properties and should be represented separately.

## 3. Shared data model

Normalize the outputs of diverse detectors into consistent records. Each observation should at minimum identify the source, event time, observation category, spatial frame, confidence, evidence pointer and producer version. A location may be pixel coordinates rather than verified GPS coordinates.

```
Observation {
  id: "obs_17", project_id: "road_a", source_id: "cam_04",
  observed_at: "2026-10-26T09:15:00+07:00",
  type: "vehicle_detected", object_class: "motorbike",
  geometry: {frame: "image_pixel", bbox: [201,88,92,64]},
  confidence: 0.93, track_id: "track_12",
```

```
  evidence_ref: "s3://.../segment.mp4#t=12.4",
  producer: {model: "detector-v1", version: "1.0"}
}
```

Raw observations should be append-only or versioned. Corrections create revisions with lineage rather than silently changing historical facts.

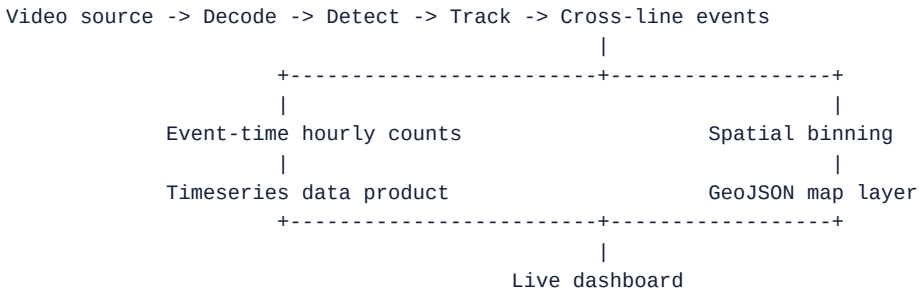
## 4. Agent roles and execution ownership

| Role | AI or deterministic? | Core responsibility |
|---|---|---|
| Perception | Vision model + worker | Detect/classify visual evidence |
| Tracking | Deterministic/model worker | Relate detections across frames |
| Spatial grounding | Calibrated geometry worker | Map image positions to known geography when possible |
| Semantic enrichment | Reasoning agent | Propose event types and relevant context |
| Product planner | Reasoning agent | Translate user intent into a pipeline specification |
| Aggregator | Deterministic worker | Count distinct tracks, group/window/measure |
| Quality verifier | Checks + optional AI | Detect invalid schemas, low coverage and unsupported claims |
| Publisher | Deterministic service | Materialize tables, APIs, map layers and dashboards |

**Non-negotiable rule:** Language models should never invent measurements from a summarized prompt. Numeric results must originate in stored observations and reproducible computations.

## 5. Product compiler: desired outcome to execution graph

A user requests: "Provide hourly motorbike counts and a live heatmap across cameras in this district." The planner first finds existing observations and published products. It then generates a typed directed acyclic graph (DAG) with reusable operations:




A pipeline definition must declare schema compatibility, dependencies, source permissions, budget limits, error handling and quality tests before it is activated. Reuse upstream outputs rather than dispatching multiple expensive video models for each new dashboard.

### Minimal product specification

```
product_id: motorbike-hourly-v1
inputs: [vehicle_crossing_events]
filter: object_class == "motorbike"
transform: count_distinct(track_id)
group_by: [camera_id, event_time:1h]
outputs: [parquet_dataset, timeseries_api, dashboard]
quality: [no_duplicate_tracks, >= 0.9 source_uptime]
refresh: continuous
owner: mobility-team
```

## 6. Continuous data: time, state and reliability

Real-world streams arrive late, out of order, duplicated or corrupted. ARGUS must distinguish the time an event was observed from the time it reached the server. Event-time windows, stateful aggregates and checkpoints are standard stream-processing approaches. [3][4]

• **Event time:** compute hourly counts using camera capture timestamps, not server arrival time.

• **Deduplication:** stable event IDs and track crossing logic prevent counting the same motorbike on every frame.

• **Late data:** accept late events within a defined lateness interval and issue corrected product versions when needed.

• **Backpressure:** sample frames or degrade gracefully without silently overstating coverage.

• **Replayability:** preserve references, models and pipeline versions to reconstruct important outputs.

### Illustrative metric contract

Motorbike flow rate for camera c and window w: **F(c,w) = number of distinct qualified crossing events / valid observation time**. Publish the number of qualifying events, actual covered duration and sampling status. This is an observed rate for that camera and geometry; do not automatically extrapolate it to an entire road network.

## 7. Concrete bridge construction example

**Inputs:** periodic drone video, fixed cameras, BIM/bridge component model, project schedule and authorized inspection records.

| Processing stage | Derived object | Potential product |
|---|---|---|
| Detect work zones and equipment | Equipment activity events | Equipment utilization summary |
| Align repeatable survey imagery | Component-level visual differences | Construction progress evidence layer |
| Flag visible surface anomalies | Candidate defect + image evidence | Inspection worklist / defect register |
| Join project schedule | Visual observations + milestones | Progress variance dashboard |
| Aggregate and explain | Verified event histories | Daily construction intelligence report |

**Safety limitation:** imagery can identify candidate visible conditions but does not prove structural capacity, hidden integrity or engineering safety. Formal inspection decisions remain with qualified professionals.

## 8. Quality, privacy and governance

• Attach source lineage, media evidence, coordinate-system metadata, model versions and transformation versions to every product.

• Use confidence and coverage separately: 95% detection confidence does not imply 95% completeness or 95% measurement accuracy.

• Avoid person identification by default. Prefer aggregate measurements, minimized retention and access controlled clips.

• Treat anomaly alerts as triage signals; require human validation for high-stakes infrastructure or safety decisions.

For interoperable sensor data, the OGC SensorThings API supplies established concepts for sensors, datastreams and observations. [5]

## 9. MVP implementation and acceptance criteria

| MVP increment | Implementation | Evidence of success |
|---|---|---|
| A. Source onboarding | One browser camera + one MP4 source | Both yield timestamped jobs |
| B. Perception | Vehicle detector and tracker | Stored typed observations and clips |
| C. Product compiler | Three approved pipeline templates | Planner creates valid typed DAGs |
| D. Stream transforms | Counts, time windows, map bins | Deduplicated output with source coverage |
| E. Publishing | Event table, chart and GeoJSON layer | Live refresh and downloadable exports |
| F. Verification | Schema/lineage tests; evidence replay | Every result traceable to source |

### Recommended stack

**Interface:** Next.js + React + MapLibre. **Ingestion:** browser camera/MP4, object storage, queue. **Workers:** Python with FFmpeg and CV inference. **State:** PostgreSQL/PostGIS for products and metadata; a stream processing service when load requires it. **Planning:** model-generated validated pipeline JSON. **Delivery:** REST/WebSocket updates and downloadable Parquet/GeoJSON/CSV.

### What to measure

• Observation quality: precision/recall on a labeled evaluation set; tracking/count error at configured boundaries.

• Pipeline performance: queue age, p95 ingest-to-product latency, throughput and dropped-frame coverage.

• Economic efficiency: GPU-seconds per video minute and compute cost per published product.

• Consumer utility: number of product subscriptions, query latency, reliability and decision review rates.

## 10. Research questions

RQ1: How much compute can a shared observation layer save versus running specialized full-video pipelines separately?

RQ2: Can an agent planner compile natural-language requests into schema-valid transformation graphs with low error rates?

RQ3: How should freshness, confidence and sampling coverage be propagated through multi-stage data products?

RQ4: When does agent reasoning improve product utility, and when are fixed transformations demonstrably safer or cheaper?

RQ5: Which product contracts allow observations from different hardware and domains to remain composable?

## 11. Conclusion

ARGUS should be engineered as a **data product compiler over a shared visual observation substrate**. The competitive capability is not simply recognizing objects in footage; it is producing and maintaining trustworthy, differentiated data outputs with transparent provenance and measurable quality. Build one complete, validated end-to-end product before scaling to dozens of domains.

## References


[1] Google Cloud, What is data as a product?. https://cloud.google.com/discover/what-is-data-as-a-product

[2] Martin Fowler, Designing Data Products. https://martinfowler.com/articles/designing-data-products.html

[3] Apache Flink, Stateful Stream Processing. https://github.com/apache/flink/blob/master/docs/content/docs/concepts/stateful-stream-processing.md

[4] Apache Flink, Event time, processing time and ingestion time. https://apache.googlesource.com/flink/+/refs/heads/blink/docs/dev/event_time.md

[5] Open Geospatial Consortium, SensorThings API. https://www.ogc.org/standards/sensorthings/